

Distributed Annealing with Exact Local Certification for Antenna Placement on Building Boundaries

Christian Inhetveen
fractal minds@protonmail.com
Technische Universität Dortmund
Dortmund, Germany

Mart Hagedoorn
mart.hagedoorn@tu-dortmund.de
Technische Universität Dortmund
Dortmund, Germany

Marcin Mennemann
marcin.mennemann@tu-dortmund.de
Technische Universität Dortmund
Dortmund, Germany

Marius Maximilian Hille
marius.hille@tu-dortmund.de
Technische Universität Dortmund
Dortmund, Germany

Kevin Buchin
kevin.buchin@tu-dortmund.de
Technische Universität Dortmund
Dortmund, Germany

Abstract

In the GIS Cup 2026, participants place k antennas on the boundaries of 50,000 building footprints such that as many buildings as possible are serviced, i.e., have at least a fraction τ of their perimeter in line of sight of an antenna. In this paper, we describe GISARD, the system behind our submission. GISARD builds initial placements greedily, improves them by simulated annealing, and distributes this search over 206 cluster nodes during the 24-hour competition. A certification stage complements this search: for a small window around each antenna, an exact integer program either finds a better position or proves that none exists within the window. Over the nine combinations of k and τ , the submission services 43,426 buildings. After 200 nodes of annealing had stalled, the certification stage still found improvements and certified almost every antenna of the final placements as locally optimal.

CCS Concepts

• **Theory of computation** → **Computational geometry**; • **Mathematics of computing** → **Combinatorial optimization**; • **Information systems** → *Geographic information systems*.

Keywords

antenna placement, visibility, art gallery problem, triangular expansion, simulated annealing, integer programming, GIS Cup

ACM Reference Format:

Christian Inhetveen, Mart Hagedoorn, Marcin Mennemann, Marius Maximilian Hille, and Kevin Buchin. 2026. Distributed Annealing with Exact Local Certification for Antenna Placement on Building Boundaries. In *The 34th ACM International Conference on Advances in Geographic Information Systems (SIGSPATIAL '26)*, November 03–06, 2026, Riverside, CA, USA. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3841645.3850167>

1 Introduction

Wireless networks at higher frequencies offer more bandwidth, but buildings block their signals. Whether a customer can be served therefore depends on the line of sight between an antenna and the customer's façade, and arranging antennas in a city becomes a geometric question. The GIS Cup, the annual programming competition of the ACM SIGSPATIAL conference, poses this question in 2026 in an idealized two-dimensional form.

Formally, we are given a set B of building footprints, i.e., polygons without holes or self-intersections in a planar map projection, a threshold $\tau \in (0, 1]$, and a budget k . A point p is *visible* from a point q if the segment pq does not intersect the interior of any building; touching a wall or a corner does not block. For a set P of antenna positions, which we call a *placement*, the *coverage* $C_P(b)$ of a building b is the total length of the parts of its boundary that are visible from at least one antenna in P . Building b is *served* if its coverage reaches its *goal* $\tau \text{ per}(b)$, where $\text{per}(b)$ denotes the perimeter of b . The task is to choose k points on building boundaries such that the number of serviced buildings, the *score*, is maximized.

Three values each for τ and k yield nine *sub-problems*, which have to be solved within 24 hours after the data are released. The organizers score each submission with their own program, the *official evaluator*, which computes in double precision and snaps an antenna within 1 mm of a building boundary onto it. Each team submits its antennas and the IDs of the buildings it claims as serviced, and a team's score on a sub-problem is normalized by the best score submitted for it. A sample dataset was available beforehand for testing; the evaluation dataset consists of 50,000 buildings with 306,833 vertices covering approximately 10.6×10.8 km, with $\tau \in \{0.32, 0.49, 0.68\}$ and $k \in \{9, 49, 484\}$.

Two properties of this problem shaped our approach. First, it is closely related to the art gallery problem, which asks for the fewest guards that together see a given polygon. For the variant in which the guards stand on the boundary and must see the boundary, Stade [9] shows that optimal guards can require irrational coordinates, even if all polygon vertices have integer coordinates. We therefore treat restricting the antennas to a finite set of candidate positions as an approximation whose quality we test empirically (Sections 3 and 7). Second, the score is a threshold count: a building contributes nothing until its coverage reaches its goal. Moving one



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGSPATIAL '26*, Riverside, CA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2950-8/2026/11

<https://doi.org/10.1145/3841645.3850167>

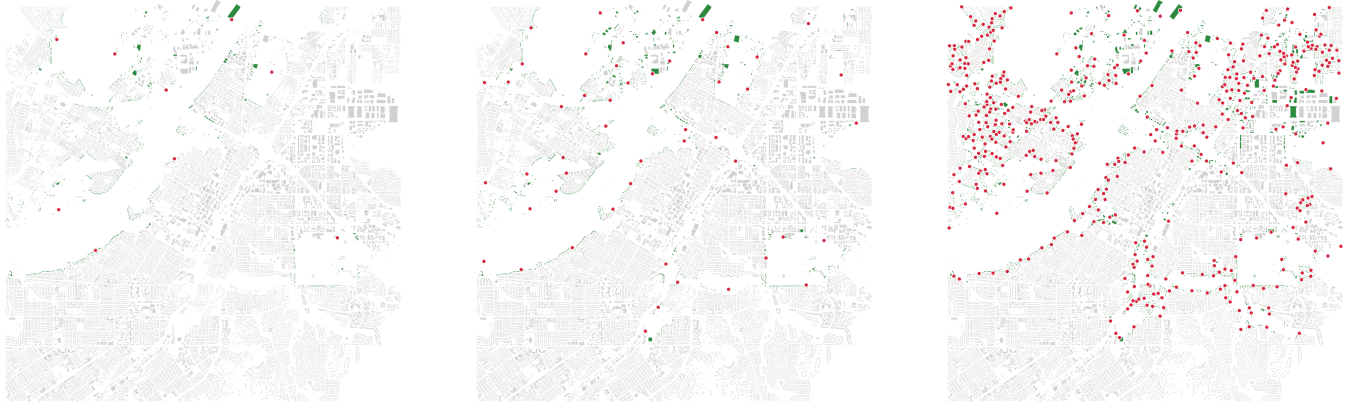


Figure 1: Submitted placements for three of the nine sub-problems (green: buildings accepted as serviced by the official evaluator; red: antennas). Left: $\tau = 0.32$, $k = 9$ (1,404 of 50,000 buildings); the antennas stand at the edges of open land, where one antenna sees hundreds of buildings. Middle: $\tau = 0.49$, $k = 49$ (2,565). Right: $\tau = 0.68$, $k = 484$ (6,059); most antennas stand in dense blocks, where the high threshold needs several antennas per building.

antenna therefore often leaves the score unchanged, and a greedy algorithm that adds one antenna at a time can stall.

Our team, the Buchineers, built GISARD in Rust. It has five stages: an exact evaluation of coverage (Section 2), a pool of candidate positions with precomputed coverage (Section 3), greedy initial placements improved by simulated annealing (Section 4), an integer program that improves a placement locally or proves that no such improvement exists (Section 5), and a campaign runner that distributes the work over a computing cluster (Section 6). The source code is available at <https://gisard.buchinee.rs>.

2 Exact visibility and coverage

Visibility queries. Since the official evaluator decides the score, we compute coverage exactly instead of sampling points along the perimeters. The *free space*, the part of the plane outside all buildings, is triangulated once into a constrained Delaunay triangulation in which every building wall is an edge. The part of the free space visible from a point q is then computed by triangular expansion (TEA) [1]: starting in the triangle that contains q , the expansion walks into each neighboring free triangle through the shared edge. It carries a *view cone*, i.e., the range of directions from q that is still unobstructed, narrows it at every crossed edge, and stops at walls, which it reports as lit wall segments. Our implementation follows the TriVis tool [5], and every wall edge additionally stores its building and its offset along that building’s perimeter, so that a single expansion returns the exact visible parts of every building’s boundary. All geometric decisions reduce to *orientation tests*, i.e., whether a point lies left of, right of, or on a line, which the adaptive predicates of Shewchuk [8] decide exactly for double-precision input. The view cones include their boundary rays, so touching a wall does not block.

Coverage as intervals. Laying the boundary of a building out flat turns its perimeter into a segment from 0 to $\text{per}(b)$, and every lit wall segment into an interval of this segment. The coverage of a building is the total length of the union of the intervals of all antennas, and whether it is serviced, the score, and every objective

used later follow from these interval sets. Positions on walls are moved 10^{-6} m outward, so that rounding cannot put them inside a building. Apart from this offset, rounding is the only source of error, and we call the result our *exact evaluator*.

3 Candidate pool and coverage matrix

All our search algorithms choose antennas from a finite *pool* of candidate positions: every polygon vertex and points at most Δ apart along each wall. A sparse *coverage matrix* stores for every candidate the buildings it sees and the intervals it covers on them, and its transpose lists for every building the candidates that see it, the building’s *viewers*.

At the start of the competition, a probe run builds matrices for decreasing Δ , starting at 64 m, until one fills the 8 GiB that we allow per coverage matrix. On the evaluation dataset, this happened at $\Delta \approx 1.6$ m with 2.2 million candidates, built in 16 s. The main run used 16, 32, and 64 times this spacing ($\Delta \approx 25.6, 51.3$, and 102.6 m), each in its own independent group of search runs, a *lane*.

4 Initial placements and simulated annealing

Surrogate objective. Since the score changes only when a building reaches its goal, we build initial placements with a smoother stand-in for it, the *surrogate* $g(P) = \sum_{b \in B} \min(\tau \text{per}(b), C_P(b))$, which credits the coverage of each building up to its goal. This function is *monotone*, i.e., adding an antenna never decreases it, and *submodular*, i.e., an antenna gains no more when added to a larger placement than when added to a smaller one. For such functions, the *greedy algorithm*, which repeatedly adds the candidate with the largest gain, attains at least a $(1 - 1/e) \approx 0.63$ fraction of the optimum [7]. Since gains only shrink as the placement grows, the lazy variant of Minoux [6] stores them in a priority queue and recomputes a gain only when its candidate reaches the top. We compute four initial placements with greedy variants that differ in what they count: the surrogate, the surrogate weighted towards buildings that can reach their goal cheaply, newly serviced buildings, and buildings that a single candidate services on its own.

Simulated annealing. We improve these placements by simulated annealing [3], a local search that repeatedly proposes a small change, a *move*, and occasionally accepts worsening moves to escape local optima. A move exchanges one antenna for a candidate outside the placement, which thus always has k antennas. The search maximizes the score plus the normalized surrogate,

$$E(P) = |\{b : C_P(b) \geq \tau \text{ per}(b)\}| + \frac{1}{2} g(P) / \sum_b \tau \text{ per}(b),$$

where the second term is at most $\frac{1}{2}$ and thus acts as a tie-break: completing a building always outweighs spreading coverage, while moves between two threshold crossings are still rewarded. A move that increases E is always accepted, and one that decreases E by δ with probability $e^{-\delta/T}$ (the Metropolis rule). The temperature T starts such that a worsening move of average size is accepted with probability 0.8 and decreases geometrically to a thousandth of this value over the time budget of the run. With probability 0.85, a move targets a building close to its goal: from a small random sample of buildings seen by the current placement, it picks the unserved one whose coverage is closest to its goal, removes the least harmful of four randomly chosen antennas, and adds the best of eight sampled viewers of that building. Each search process, which we call a *worker*, runs four independent annealing runs from different initial placements on four cores and keeps the best placement that any of them visited.

5 Certified window repair

Proving a whole placement optimal is out of reach, but we can prove that no small change to it improves the score. We formulate the problem as an integer linear program (ILP) and restrict it to small windows of a placement.

Atoms. Fix a building b and the candidate pool C . The endpoints of all intervals that candidates in C cover on the boundary of b cut its perimeter into elementary pieces, which we call *atoms*. Let A_b be the atoms of b , ℓ_a the length of atom a , and $C_a \subseteq C$ the candidates that see it. By construction, a candidate sees an atom either completely or not at all. Hence, the coverage of b under a placement $S \subseteq C$ becomes a plain sum instead of the length of a union of intervals: the sum of ℓ_a over the atoms with $C_a \cap S \neq \emptyset$.

Integer program. A binary variable x_c states whether candidate c is chosen, a binary variable y_b whether building b counts as serviced, and a variable $z_a \in [0, 1]$ whether atom a counts as covered. The problem then reads

$$\begin{aligned} \max \quad & \sum_{b \in B} y_b \\ \text{s.t.} \quad & \sum_{c \in C} x_c \leq k, \end{aligned} \quad (1)$$

$$z_a \leq \sum_{c \in C_a} x_c \quad \forall b \in B, a \in A_b, \quad (2)$$

$$\sum_{a \in A_b} \ell_a z_a \geq \tau \text{ per}(b) y_b \quad \forall b \in B. \quad (3)$$

Constraint (1) enforces the budget, Constraint (2) lets an atom count as covered only if a chosen candidate sees it, and Constraint (3) lets a building count as serviced only if its covered atoms reach its goal. Since the objective pushes every y_b up, the optimal value equals the maximum score. The z_a need not be declared integral: the right-hand side of (2) is an integer, so some optimal solution has $z_a \in \{0, 1\}$.

An ILP solver proves a solution optimal by comparing it with an upper bound from the *linear relaxation*, i.e., the same program with

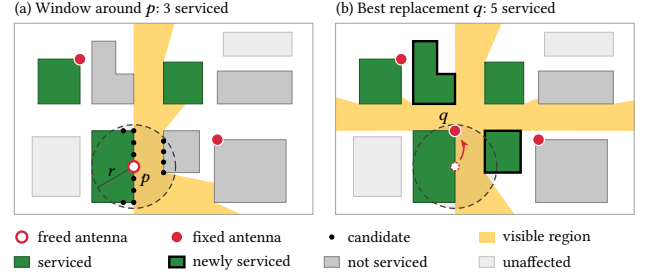


Figure 2: A window on a toy instance ($\tau = 0.45$). (a) The antenna p is freed; the candidates lie within radius r of it. (b) The ILP picks the best candidate q , which services two more buildings.

all variables allowed to be fractional. For full instances, we tighten this bound with four families of *valid inequalities*, i.e., constraints that all integral solutions satisfy but some fractional ones violate. The simplest states that a serviced building needs at least as many chosen viewers as the fewest viewers whose coverages sum to its goal; the others are capped-coverage, submodular, and knapsack-cover inequalities. We solve the ILP with the open-source solver HiGHS [2]; with one variable per atom, it handles at most a few hundred buildings.

Windows. We therefore apply the ILP to small *windows* of the incumbent P , the best placement found so far (Figure 2). A window frees one antenna $p \in P$ and keeps all others fixed. The candidates for replacing p are the pool positions within a radius r of p and p itself, so that the ILP can always keep P unchanged. Only the *affected* buildings, i.e., those seen by p or by one of these candidates, can change their status. Of these, the buildings that the fixed antennas already service drop out, and every other one keeps a *residual goal*, i.e., its goal minus what the fixed antennas cover; only the uncovered parts of the candidates' intervals count towards it. The ILP (1)–(3) with budget one then maximizes the number of residual goals reached. If it services more affected buildings than P does, the replacement improves the whole placement, since no other building changes. Otherwise, the window is *certified*: no candidate within the disc services more buildings than p . The certificate is relative to the pool and the disc; for one freed antenna, it amounts to an exhaustive check. Improvements from windows with disjoint affected buildings are combined and kept only if re-scoring the whole placement confirms them. Since the verdict of a window depends only on the incumbent, windows can run on different cluster nodes without coordination.

6 The 24-hour campaign

We ran the search on the LiDO3 cluster of TU Dortmund University, whose nodes have 20 cores and 64 GB of memory. Annealing used 200 nodes with five workers per node and one coverage matrix per worker. For comparison, six nodes ran a large neighborhood search (LNS), which repeatedly removes a few antennas and greedily re-inserts them, for the first eight hours and then switched to window repair. After the annealing workers had stopped, one node each ran a final round of window repair and the harvest, for a total of 206 nodes, 4,120 cores, and about 93,000 core-hours.



Figure 3: Zoomed views with the regions visible from the antennas (orange; scale bars: 100 m). Left: $\tau = 0.32$, $k = 9$, an antenna on a corner facing open land. Middle: $\tau = 0.49$, $k = 49$, an antenna at a street crossing. Right: $\tau = 0.68$, $k = 484$, a dense block where several regions overlap to reach the high threshold.

Table 1: Buildings serviced per sub-problem (official evaluator; 50,000 buildings in total).

$\tau \backslash k$	9	49	484	row
0.32	1,404	4,056	16,300	21,760
0.49	819	2,565	10,756	14,140
0.68	260	1,207	6,059	7,526
total	2,483	7,828	33,115	43,426

The work is split into *trials*, each an annealing run of one worker for a given sub-problem, lane, random seed, and time budget. Trials are scheduled by successive halving [4], which runs many trials briefly and continues only the most promising ones: per sub-problem and lane, groups of 64 trials run for 15 minutes, and the best quarter of each group is continued to one hour, and the best quarter of those to four hours, resuming from a checkpoint of its best placement.

In parallel, window repair runs in *waves*, each of which checks a window around every antenna of every sub-problem. When a wave finds no improvement, the next one uses stronger windows: longer solver time limits, radii of 120 and 200 m, and finally two freed antennas; an improvement resets this sequence. The final hour is reserved for the *harvest*, which re-evaluates the best checkpoint of each sub-problem and checks that the submission file preserves every coordinate; the official evaluator then derives the claimed building IDs.

Use of AI tools. Claude Code (Opus 4.8) assisted with the implementation of the solver, which let us try out new approaches quickly. We designed the methods, tested the solver extensively, and take full responsibility for the code and results.

7 Results

Table 1 lists the buildings accepted as serviced by the official evaluator per sub-problem. Our exact evaluator scores the same placements at 43,402 buildings; the official evaluator accepts 24 more, since its tolerance-based geometry admits some borderline buildings. Figures 1 and 3 show examples.

Contribution of the stages. Annealing on 200 nodes found its last improvement within the first six hours of the run. Every later improvement came from window repair on six nodes, which

gained 26 buildings in total. Most came from windows of radius 60 m, whereas larger windows found little and two freed antennas nothing, so the annealing placements were nearly locally optimal.

Annealing versus LNS. LNS reached 98% of its final score within five minutes on 30 workers, compared to 15 minutes on 1,000 workers for annealing, but then stalled about 1.7% below it.

Candidate density. The candidate spacing mattered little: on the sample dataset, a four times finer spacing increased the number of buildings that a single candidate can service by 0.38%, and on a small instance cut out of it, the optimum proven by the ILP is identical for $\Delta \in \{8, 4, 2, 1\}$ m, presumably because the decisive positions lie near corners.

8 Discussion

Computing coverage exactly kept our internal score within 24 buildings (0.06%) of the official one. The campaign also shows the limits of search at this scale: 200 nodes of annealing plateaued long before the deadline, and the remaining gains came from the ILP on six nodes, which at radius 60 m also certified all but one antenna of the final placements.

Future Work. Our certificates are local and relative to the candidate pool. A global upper bound over the pool would show how far the placements are from optimal, and stronger formulations could certify windows with several freed antennas.

Acknowledgments

The authors gratefully acknowledge the computing time provided on the Linux HPC cluster at Technical University Dortmund (LiDO3), partially funded in the course of the Large-Scale Equipment Initiative by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) as project 271512359. We especially thank Sven Buijssen for reserving LiDO3 nodes for the competition and for his friendly and prompt help, even late at night.

References

- [1] Francis Bungiu, Michael Hemmer, John Hersberger, Kan Huang, and Alexander Kröller. 2014. Efficient Computation of Visibility Polygons. *arXiv preprint arXiv:1403.3905* (2014).
- [2] Qi Huangfu and J. A. Julian Hall. 2018. Parallelizing the Dual Revised Simplex Method. *Mathematical Programming Computation* 10, 1 (2018), 119–142. doi:10.1007/s12532-017-0130-5
- [3] Scott Kirkpatrick, C. Daniel Gelatt, and Mario P. Vecchi. 1983. Optimization by Simulated Annealing. *Science* 220, 4598 (1983), 671–680. doi:10.1126/science.220.4598.671
- [4] Liam Li, Kevin Jamieson, Afshin Rostamizadeh, Ekaterina Gonina, Jonathan Bentzur, Moritz Hardt, Benjamin Recht, and Ameet Talwalkar. 2020. A System for Massively Parallel Hyperparameter Tuning. In *Proc. Conference on Machine Learning and Systems (MLSys '20)*. 230–246.
- [5] Jan Mikula, Miroslav Kulich, and Libor Přeučil. 2024. TriVis: Versatile, Reliable, and High-Performance Tool for Computing Visibility in Polygonal Environments. In *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS '24)*. 10503–10510. doi:10.1109/IROS58592.2024.10801476
- [6] Michel Minoux. 1978. Accelerated Greedy Algorithms for Maximizing Submodular Set Functions. In *Optimization Techniques. Lecture Notes in Control and Information Sciences*, Vol. 7. Springer, 234–243. doi:10.1007/BFb0006528
- [7] George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. 1978. An Analysis of Approximations for Maximizing Submodular Set Functions—I. *Mathematical Programming* 14, 1 (1978), 265–294. doi:10.1007/BF01588971
- [8] Jonathan Richard Shewchuk. 1997. Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates. *Discrete & Computational Geometry* 18, 3 (1997), 305–363. doi:10.1007/PL00009321
- [9] Jack Stade. 2026. NP-Membership for the Boundary-Boundary Art-Gallery Problem. In *Proc. ACM Symposium on Theory of Computing (STOC '26)*. 256–267. doi:10.1145/3798129.3800745